

1 Key Ideas

We present our key ideas for accelerating the MIP-based computation of $\hat{\delta}_{ct}$. For tightening the MIP’s LP relaxation, we use *Perturbed-interval constraints* and *zonotope bound tightening*. Furthermore, we relax binaries of near-decided neurons using *triangle relaxation over two networks*, and using a *warm start from a previously verified network* N_{pre} for faster convergence of a target network N .

Perturbed-Interval Constraints. We rely on VHAGaR’s two-copy MIP encoding of the network: a clean copy on input $x \in [0, 1]^n$ and a perturbed copy on $x_p = x + v_e$ with $v_e \in [-\varepsilon, \varepsilon]^n$. We introduce tight bounds between the two copies via *perturbed-interval constraints* that prune the search space and improve the running times. Concretely, for each ReLU neuron (m, k) shared by the two copies, we add the two linear inequalities

$$z_{m,k}^{\text{pert}} \leq z_{m,k}^{\text{org}} + \Delta_{m,k}^{\uparrow}, \quad (1)$$

$$z_{m,k}^{\text{pert}} \geq z_{m,k}^{\text{org}} + \Delta_{m,k}^{\downarrow}, \quad (2)$$

where $[\Delta_{m,k}^{\downarrow}, \Delta_{m,k}^{\uparrow}]$ is a sound interval over-approximation of the per-neuron post-ReLU drift. No new variables are introduced; nothing inside either copy’s encoding changes.

We compute $[\Delta_{m,k}^{\downarrow}, \Delta_{m,k}^{\uparrow}]$ via a layer-by-layer interval recurrence over the network. We initialize at the input with the perturbation bound $[\Delta_0^{a,\downarrow}, \Delta_0^{a,\uparrow}] = [-\varepsilon \mathbf{1}, +\varepsilon \mathbf{1}]$. For an affine layer with weight W , the bias cancels and interval arithmetic on $W^\top \Delta_{m-1}^a$ gives the pre-activation drift bounds:

$$\begin{aligned} \Delta_{m,k}^{z,\downarrow} &= \sum_{k'} \min(W_{k',k} \Delta_{m-1,k'}^{a,\downarrow}, W_{k',k} \Delta_{m-1,k'}^{a,\uparrow}), \\ \Delta_{m,k}^{z,\uparrow} &= \sum_{k'} \max(W_{k',k} \Delta_{m-1,k'}^{a,\downarrow}, W_{k',k} \Delta_{m-1,k'}^{a,\uparrow}). \end{aligned} \quad (3)$$

For a ReLU layer, the pointwise identity $\sigma(z + \Delta) - \sigma(z) \in [\min(0, \Delta), \max(0, \Delta)]$ yields

$$\Delta_{m,k}^{a,\downarrow} = \min(0, \Delta_{m,k}^{z,\downarrow}), \quad \Delta_{m,k}^{a,\uparrow} = \max(0, \Delta_{m,k}^{z,\uparrow}). \quad (4)$$

Example. Consider an ℓ_∞ perturbation with $\varepsilon = 0.1$ on a network whose first hidden ReLU neuron k at layer $m = 1$ has incoming weight row $(w_{1,k}, w_{2,k}) = (1, -2)$. The input bound is $\Delta_0^a \in [-0.1, 0.1]^2$. By (3), $\Delta_{1,k}^{z,\uparrow} = 0.1 + 0.2 = 0.3$ and $\Delta_{1,k}^{z,\downarrow} = -0.1 - 0.2 = -0.3$; (4) then yields $[\Delta_{1,k}^{a,\downarrow}, \Delta_{1,k}^{a,\uparrow}] = [-0.3, 0.3]$. We therefore add to the MIP the constraints $z_{1,k}^{\text{pert}} - z_{1,k}^{\text{org}} \in [-0.3, 0.3]$, coupling the two copies’ post-ReLU activations at this neuron. Figure 1 illustrates this propagation, with the weights on the shared edges and the per-edge arithmetic.

Relocation-Aware Perturbed Intervals. The recurrence above models the perturbed input as the clean input plus per-pixel noise, $x_p = x + v_e$ with $v_e \in [-\varepsilon, \varepsilon]^n$. This is the correct model for ℓ_∞ and brightness, but not for geometric perturbations such as translation and rotation. A geometric perturbation does not add noise to a pixel; it *relocates* pixel values, so the intensity at one location is copied (translation) or blended (rotation) from other locations, and locations that fall outside the frame become black. Initializing the recurrence with the generic input box $[-\varepsilon \mathbf{1}, +\varepsilon \mathbf{1}]$ is then both meaningless and wasteful, because for a move ε is a shift count or a rotation angle rather than a bound on how much a pixel intensity can change. We instead feed the recurrence the exact algebraic structure of the move, which yields far tighter drift bounds while leaving $\hat{\delta}_{ct}$ unchanged.

A translation and a rotation are both linear maps of the clean image. Each is a fixed matrix $T \in \mathbb{R}^{n \times n}$ acting on the input, so the perturbed input is $x_p = Tx$ and the input drift is exactly

$$\Delta_0 = x_p - x = (T - I)x, \quad (5)$$

a known linear function of the clean image rather than a free box. For a translation by (s_d, s_r) pixels, $T = S$ is the shift operator: row d of S has a single 1 at the source pixel that lands on d , or is all zeros when d falls outside the frame, so x_p either copies one clean pixel into d or sets d to 0. For a rotation by an angle θ , $T = R$ is the bilinear sampler: row d of R holds the four bilinear interpolation weights of the rotated source location of d , which are nonnegative and sum to 1, or is all zeros when that location is outside the frame.

We exploit (5) by folding $(T - I)$ into the first affine layer instead of seeding the recurrence at the input. With first-layer weight W_1 , the shared bias cancels in the difference and the first pre-activation drift is the single linear map $\Delta_1^z = W_1^\top (T - I)x = Mx$, where $M = W_1^\top (T - I)$. Because each clean pixel ranges independently over $x_j \in [0, 1]$, the exact range of each coordinate $(Mx)_k = \sum_j M_{k,j} x_j$ is attained at a box corner, which gives the closed form

$$\Delta_{1,k}^{z,\downarrow} = \sum_j \min(0, M_{k,j}), \quad \Delta_{1,k}^{z,\uparrow} = \sum_j \max(0, M_{k,j}). \quad (6)$$

From the second layer onward nothing changes: we apply the same affine recurrence (3) and ReLU clip (4), and we add the same coupling constraints (1) and (2). As a concrete case, a right shift by s pixels sets $x_p[d] = x[d - s]$, so the drift $\Delta_0[d] = x[d - s] - x[d]$ lies in $[-1, 1]$, bounded by the pixel intensities alone and far tighter than the generic per-pixel seed $[-s, s]$ that grows with the shift. This composition applies whenever the first layer is fully connected; a convolutional first layer is also linear, but our implementation then falls back to the generic recurrence.

Soundness. The relocation-aware bounds are sound: the interval $[\Delta_{m,k}^\downarrow, \Delta_{m,k}^\uparrow]$ over-approximates the true post-ReLU drift of neuron (m, k) for every clean image $x \in [0, 1]^n$, so the constraints (1) and (2) cut no feasible point and the computed $\hat{\delta}_{c_t}$ is unchanged. We prove this separately for translation and rotation. In both cases the first-layer range (6) is in fact exact, and the deeper ReLU and affine steps are sound interval enclosures by the same argument as the generic recurrence, so it suffices to establish $x_p = Tx$ and the exactness of (6).

Translation. The encoder pins each perturbed pixel to a clean pixel: $x_p[d] = x[\text{src}(d)]$ when the destination d is in frame, and $x_p[d] = 0$ when d is padded. Both cases are exactly row d of S applied to x , so $x_p = Sx$ and $\Delta_0 = (S - I)x$ hold with equality rather than as a relaxation. Hence the first pre-activation drift is the exact linear map Mx with $M = W_1^\top (S - I)$, and since each x_j varies independently over $[0, 1]$, the minimum and the maximum of $\sum_j M_{k,j} x_j$ are attained by setting $x_j = 1$ exactly on the negative and the positive entries of row k , which is (6). The first-layer interval is therefore the tight range of the drift, while the ReLU clip (4) and the deeper affine steps (3) are sound enclosures, so the final per-neuron interval contains the true drift for every $x \in [0, 1]^n$. The coupling constraints thus hold at every feasible MIP point, and $\hat{\delta}_{c_t}$ is preserved.

Rotation. The encoder pins each perturbed pixel to the bilinear blend of its rotated source: $x_p[d] = \sum_q R_{d,q} x[q]$ with weights $R_{d,q} \geq 0$ that sum to 1 when the source is in frame, and $x_p[d] = 0$ when it is out of frame. Both cases are exactly row d of R applied to x , so $x_p = Rx$ and $\Delta_0 = (R - I)x$ hold with equality. The rest of the argument is identical to the translation case with S replaced by R : the first-layer range (6) with $M = W_1^\top (R - I)$ is exact because each $x_j \in [0, 1]$ independently, and the subsequent ReLU and affine steps are sound enclosures. Moreover, each bilinear weight lies in $[0, 1]$ and every in-frame row of R sums to 1, so each perturbed pixel stays in $[0, 1]$ and matches the encoder’s variable bounds, which means no admissible rotation is excluded. Hence the constraints are sound and $\hat{\delta}_{c_t}$ is preserved.

Zonotope Bound Tightening. The MIP encodes each ReLU neuron (m, k) with big- M constants $\underline{z}_{m,k}, \bar{z}_{m,k}$ derived from per-neuron pre-activation bounds $[l_{m,k}, u_{m,k}]$. Loose bounds inflate the feasible region of every LP relaxation solved at the branch-and-bound nodes, and they keep binary variables alive on neurons that are already stable under all admissible inputs. Tighter bounds therefore shrink the search space directly, and they may flip a split neuron ($l_{m,k} < 0 < u_{m,k}$) to stable, dropping its binary $a_{m,k}$ from the MIP entirely. We obtain such tighter bounds by propagating a zonotope through the network and reading per-neuron hull bounds at each ReLU layer, which we then inject as the big- M constants of the encoding.

A zonotope represents each pre-activation $\hat{z}_{m,k}$ as a centre plus a linear combination of shared noise variables $\eta_j \in [-1, 1]$ (distinct from the perturbation budget ε), stored as a centre vector $\mathbf{c}_m \in \mathbb{R}^K$ and a generator matrix $G_m \in \mathbb{R}^{K \times p}$:

$$\hat{z}_{m,k} = c_{m,k} + \sum_{j=1}^p g_{m,k,j} \eta_j, \quad \eta_j \in [-1, 1]. \quad (7)$$

Sharing η_j across neurons of the same layer preserves correlations, so a downstream affine layer contributes the tighter half-width $|\sum_{k'} w_{k,k'} g_{m-1,k',j}|$ rather than the looser interval sum $\sum_{k'} |w_{k,k'} g_{m-1,k',j}|$ (with equality only when all terms share a sign). We initialise the zonotope so its hull covers both the clean input $v_{\text{in}} \in [0, 1]^n$ and the perturbed input $v_{x_0} = v_{\text{in}} + v_e \in [-\varepsilon, 1 + \varepsilon]^n$ simultaneously. Let $r = \frac{1}{2} + \varepsilon$ denote the per-coordinate half-width of the widened input box $[-\varepsilon, 1 + \varepsilon]^n$; then

$$\mathbf{c}_0 = \frac{1}{2} \mathbf{1}, \quad G_0 = r I_n \in \mathbb{R}^{n \times n}, \quad (8)$$

so a single pass through the network bounds the pre-activations under both copies at once.

For an affine layer with weight W and bias b , the zonotope propagates exactly: $\mathbf{c} \leftarrow W\mathbf{c} + b$ and $G \leftarrow WG$. For a ReLU layer, we apply the standard DeepZ relaxation [1], with per-neuron hull endpoints

$$l_k = c_{m,k} - \sum_j |g_{m,k,j}|, \quad u_k = c_{m,k} + \sum_j |g_{m,k,j}|. \quad (9)$$

Stable neurons ($u_k \leq 0$ or $l_k \geq 0$) pass through unchanged or are zeroed; split neurons ($l_k < 0 < u_k$) are scaled by $\lambda_k = u_k / (u_k - l_k)$ and offset by $\mu_k = -u_k l_k / (2(u_k - l_k))$:

$$c_{m,k} \leftarrow \lambda_k c_{m,k} + \mu_k, \quad g_{m,k,\cdot} \leftarrow \lambda_k g_{m,k,\cdot}, \quad (10)$$

with a new generator column appended that carries μ_k at row k and zero elsewhere. At every ReLU layer we extract the per-neuron hull (l_k, u_k) from Eq. (9) *before* the relaxation overwrites the row, and inject these as the tightened pre-activation interval $[l_{m,k}, u_{m,k}]$ for the big- M constants on neuron (m, k) .

When a prior verification of a structurally similar network N_{pre} is available, we tighten N 's bounds further by propagating the weight drift between N_{pre} and N through the same zonotope abstraction.

Given N_{pre} 's per-neuron bounds and the weight and biases differences $\Delta W = W - W_{\text{pre}}, \Delta B = B - B_{\text{pre}}$, we compute a sound interval $[d_{m,k}^\downarrow, d_{m,k}^\uparrow]$ on the per-neuron pre-activation difference $\hat{z}_{m,k}^N - \hat{z}_{m,k}^{N_{\text{pre}}}$ by propagating ΔW and ΔB through the network. Combined with N_{pre} 's known bounds, this yields tighter pre-activation bounds for N :

$$l_{m,k}^N \geq l_{m,k}^{N_{\text{pre}}} + d_{m,k}^\downarrow, \quad u_{m,k}^N \leq u_{m,k}^{N_{\text{pre}}} + d_{m,k}^\uparrow. \quad (11)$$

We propagate ΔW and ΔB from the previous layer to calculate the differences for current layer m . Writing $\Delta w_{m,k,k'} = w_{m,k,k'} - w_{m,k,k'}^{\text{pre}}$ and $\Delta b_{m,k} = b_{m,k} - b_{m,k}^{\text{pre}}$ for the per-edge weight and bias

drift, the decomposition at an affine layer is

$$\begin{aligned}\hat{z}_{m,k}^N - \hat{z}_{m,k}^{N_{\text{pre}}} &= \sum_{k'} w_{m,k,k'} (z_{m-1,k'}^N - z_{m-1,k'}^{N_{\text{pre}}}) \\ &\quad + \sum_{k'} \Delta w_{m,k,k'} z_{m-1,k'}^{N_{\text{pre}}} + \Delta b_{m,k},\end{aligned}\tag{12}$$

We bound it with a zonotope abstraction, in the same spirit as the absolute zonotope above, but applied to the diff. We represent the per-neuron diff $d_{m,k} \triangleq \hat{z}_{m,k}^N - \hat{z}_{m,k}^{N_{\text{pre}}}$ as a centre plus a linear combination of shared noise variables $\eta_j \in [-1, 1]$, stored as a centre vector $\mathbf{c}_m \in \mathbb{R}^K$ and a generator matrix $G_m \in \mathbb{R}^{K \times p}$ with per-neuron hull.

$$[d_{m,k}^\downarrow, d_{m,k}^\uparrow] = [c_{m,k} - \sum_j |g_{m,k,j}|, c_{m,k} + \sum_j |g_{m,k,j}|].\tag{13}$$

Both networks receive the same input, so we initialise at the input with zero diff and no generators, $\mathbf{c}_0 = \mathbf{0}_n$ and $G_0 = \mathbf{0}_{n \times 0}$. The affine update is evaluated at the midpoint mid_{m-1} and half-width rad_{m-1} of N_{pre} 's post-ReLU range $R_{m-1,k'}^{N_{\text{pre}}} \triangleq [\underline{a}_{m-1,k'}, \bar{a}_{m-1,k'}]$, where $\underline{a}_{m-1,k'} \triangleq \max(0, l_{m-1,k'}^{N_{\text{pre}}})$ and $\bar{a}_{m-1,k'} \triangleq \max(0, u_{m-1,k'}^{N_{\text{pre}}})$ are the ReLU-clipped pre-activation bounds of N_{pre} :

$$\begin{aligned}\text{mid}_{m-1,k'} &= \frac{1}{2}(\bar{a}_{m-1,k'} + \underline{a}_{m-1,k'}), \\ \text{rad}_{m-1,k'} &= \frac{1}{2}(\bar{a}_{m-1,k'} - \underline{a}_{m-1,k'}),\end{aligned}\tag{14}$$

stacked over k' into the vectors mid_{m-1} and rad_{m-1} used below. The centre then carries the propagated diff centre plus the direct weight-drift contribution evaluated at this midpoint, and the generator matrix concatenates the propagated generators with a fresh diagonal block of independent generators scaled by ΔW and the post-ReLU half-widths:

$$\begin{aligned}\mathbf{c}_m &= W \mathbf{c}_{m-1} + \Delta W \cdot \text{mid}_{m-1} + \Delta b_m, \\ G_m &= [W G_{m-1} \mid \Delta W \cdot \text{diag}(\text{rad}_{m-1})],\end{aligned}\tag{15}$$

At a ReLU layer, the per-neuron hull from Eq. (13) drives the same DeepZ relaxation of Eq. (10), and we read $[d_{m,k}^\downarrow, d_{m,k}^\uparrow]$ at each ReLU site for use in Eq. (11).

The diff zonotope bounds N relative to N_{pre} . We tighten further with a second, independent zonotope that bounds N 's *absolute* pre-activations, propagating through N alone, with the input-covering initialisation of Eq. (8), the affine and DeepZ updates, and the per-neuron hull of Eq. (9) all exactly as above, only applied to N rather than to the two-copy encoding. We write $[l_{m,k}^{\text{zono}(N)}, u_{m,k}^{\text{zono}(N)}]$ for the resulting hull of Eq. (9) on N 's absolute pre-activation $\hat{z}_{m,k}^N$, and intersect it with the diff-based bound of Eq. (11) by keeping the tighter of the two at every neuron:

$$\begin{aligned}l_{m,k}^N &= \max(l_{m,k}^{\text{zono}(N)}, l_{m,k}^{N_1} + d_{m,k}^\downarrow), \\ u_{m,k}^N &= \min(u_{m,k}^{\text{zono}(N)}, u_{m,k}^{N_1} + d_{m,k}^\uparrow).\end{aligned}\tag{16}$$

We then propagate these intersected bounds through the absolute zonotope's DeepZ relaxation, so each layer passes through a narrower tube.

Both zonotopes are sound abstract interpretations, so each per-neuron hull contains every realisable value of its quantity. The intersection in Eq. (16) of two sound over-approximations is itself sound.

Linear Abstraction Over Two Networks. For each split ReLU neuron with pre-activation bounds $[l, u]$, the area of its triangle LP relaxation is $\text{gap}(l, u) = u|l|/(2(u-l))$. Per-copy triangle drop replaces the binary a and its big- M constraints with the three triangle inequalities on every (neuron, copy) whose gap is below a user-specified threshold τ , leaving the other neurons with their exact encoding. The result is a sound upper bound $\delta \geq \delta^*$ that approaches δ^* as τ decreases (equality at $\tau = 0$).

Per-copy triangle drop drops the binary on a copy whenever that copy’s gap is below the threshold τ . Writing $g^o \triangleq \text{gap}(l_{m,k}^{\text{org}}, u_{m,k}^{\text{org}})$ and $g^p \triangleq \text{gap}(l_{m,k}^{\text{pert}}, u_{m,k}^{\text{pert}})$ for the two per-copy gaps (the (m, k) indices are implicit), every split ReLU neuron falls into one of four tiers:

$$\text{tier} = \begin{cases} \text{both-thin}, & g^o, g^p < \tau, \\ \text{one-thin}^{\text{org}}, & g^o < \tau \leq g^p, \\ \text{one-thin}^{\text{pert}}, & g^p < \tau \leq g^o, \\ \text{neither}, & g^o, g^p \geq \tau. \end{cases} \quad (17)$$

The superscript on one-thin^{org} (resp. one-thin^{pert}) marks which copy is relaxed. In the both-thin tier, per-copy triangle drop drops both binaries; in either one-thin tier, it drops only the relaxed copy’s binary and keeps the sibling’s binary exact; in the neither tier, both binaries stay.

The relaxation augments per-copy triangle drop’s emission in two ways. In the one-thin tier, the relaxed copy’s simple triangle ignores the surviving sibling binary on the kept copy, even though that binary carries an exact $\{0, 1\}$ value. Conditioning the relaxed copy’s envelope on this sibling binary yields a strictly tighter LP relaxation, since each sibling branch forces a sub-interval of the unconditional pre-activation bound on the relaxed copy; we therefore replace the simple triangle with two big- M -gated conditional triangles, one per sibling branch. In the both-thin tier, we additionally inject a single pre-activation coupling line linking the two relaxed copies.

We write the one-thin equations for the org-relaxed sub-case; the pert-relaxed sub-case is the mirror image, with org and pert swapped throughout. From per-copy pre-activation bounds $[l_{m,k}^{\text{org}}, u_{m,k}^{\text{org}}]$ and $[l_{m,k}^{\text{pert}}, u_{m,k}^{\text{pert}}]$ (see Zonotope Bound Tightening) and the pre-activation drift interval $[\Delta_{m,k}^{z,\downarrow}, \Delta_{m,k}^{z,\uparrow}]$ for $\hat{z}_{m,k}^{\text{pert}} - \hat{z}_{m,k}^{\text{org}}$ (Eq. (3), read at the pre-ReLU site, before the clip of Eq. (4)), we compute the conditional bounds

$$l_A = \max(-\Delta_{m,k}^{z,\uparrow}, l_{m,k}^{\text{org}}), \quad (18)$$

$$u_A = \min(u_{m,k}^{\text{pert}} - \Delta_{m,k}^{z,\downarrow}, u_{m,k}^{\text{org}}), \quad (19)$$

$$l_I = \max(l_{m,k}^{\text{pert}} - \Delta_{m,k}^{z,\uparrow}, l_{m,k}^{\text{org}}), \quad (20)$$

$$u_I = \min(-\Delta_{m,k}^{z,\downarrow}, u_{m,k}^{\text{org}}). \quad (21)$$

Each pair is a strict sub-interval of $[l_{m,k}^{\text{org}}, u_{m,k}^{\text{org}}]$: $[l_A, u_A]$ is the active-sibling branch ($a_{m,k}^{\text{pert}} = 1$, so $\hat{z}_{m,k}^{\text{pert}} \geq 0$), and $[l_I, u_I]$ is the inactive-sibling branch ($a_{m,k}^{\text{pert}} = 0$, so $\hat{z}_{m,k}^{\text{pert}} \leq 0$).

On the relaxed org copy, we replace per-copy triangle drop’s simple triangle with the four inequalities

$$z_{m,k}^{\text{org}} \geq 0, \quad z_{m,k}^{\text{org}} \geq \hat{z}_{m,k}^{\text{org}}, \quad (22)$$

$$\begin{aligned} z_{m,k}^{\text{org}} &\leq \frac{u_A}{u_A - l_A} (\hat{z}_{m,k}^{\text{org}} - l_A) \\ &\quad + M(1 - a_{m,k}^{\text{pert}}), \end{aligned} \quad (23)$$

$$\begin{aligned} z_{m,k}^{\text{org}} &\leq \frac{u_I}{u_I - l_I} (\hat{z}_{m,k}^{\text{org}} - l_I) \\ &\quad + M a_{m,k}^{\text{pert}}, \end{aligned} \quad (24)$$

with $M = u_{m,k}^{\text{org}} + |l_{m,k}^{\text{org}}|$ a big- M slack chosen large enough to make the off-branch upper bound vacuous over the feasible region. When $a_{m,k}^{\text{pert}} = 1$, Eq. (23) enforces the tighter $[l_A, u_A]$ triangle while Eq. (24) is slack; the roles swap when $a_{m,k}^{\text{pert}} = 0$. No new variables are introduced beyond the existing $z_{m,k}^{\text{org}}$ and $\hat{z}_{m,k}^{\text{org}}$, since the surviving binary $a_{m,k}^{\text{pert}}$ already lives in the pert copy’s exact big- M encoding.

For a both-thin neuron, both copies receive the simple triangle of per-copy triangle drop and we additionally emit one pre-activation coupling line,

$$\Delta_{m,k}^{z,\downarrow} \leq \hat{z}_{m,k}^{\text{pert}} - \hat{z}_{m,k}^{\text{org}} \leq \Delta_{m,k}^{z,\uparrow}, \quad (25)$$

the pre-ReLU analogue of the post-activation coupling in Eqs. (1)–(2), which prevents the LP relaxation from choosing the two copies’ pre-activations independently. Eqs. (19)–(21) intersect two independently sound sources, so on whichever sibling branch the binary lands the gated triangle in Eqs. (23)–(24) still envelopes the exact ReLU output. The resulting bound $\delta \geq \delta^*$ is no looser than per-copy triangle drop’s, and strictly tighter on every node of Gurobi’s branch tree that fixes the surviving sibling binary when the two sources disagree.

So far we drove per-copy triangle drop and sibling-gated refinement with N ’s own per-neuron bounds. When a prior verification of a structurally similar network N_{pre} is available, the techniques are unchanged, but the per-copy bounds $[l_{m,k}^{N,\text{copy}}, u_{m,k}^{N,\text{copy}}]$, $\text{copy} \in \{\text{org}, \text{pert}\}$, that drive their gap test become N ’s *transfer-tightened* bounds from Zonotope Bound Tightening (Eqs. (11) and (16)): the intersection of N_{pre} ’s reused per-copy bound plus the weight-drift diff with N ’s absolute zonotope hull. This changes the technique in three ways:

- **Which binaries drop.** The gap test $\text{gap}(l_{m,k}^{N,\text{copy}}, u_{m,k}^{N,\text{copy}}) < \tau$ is evaluated on the bounds we actually write into N ’s MIP, so it inherits the tightening that N_{pre} ’s prior verification provides. As the transfer bounds are contained in N ’s standalone zonotope hull, at least as many neurons qualify as near-decided.
- **The org/pert asymmetry.** The two copies’ gaps can differ because N_{pre} ’s reused per-copy bounds differ: the perturbed copy inherits a wider input box, so a neuron may be thin on one copy and fat on the other even where the weight-drift diff is identical.
- **The conditional triangles.** Sibling-gated refinement builds its gated envelopes from these same transfer-tightened per-copy bounds, intersected with the org/pert pre-activation drift propagated through N .

Suboptimal Lower Bound and Variable Hints. We apply VHAGAR’s variable hinting technique to N ’s MIP and suboptimal lower bound, with a modification for transfer mode. We run a pgd optimization process to compute a lower bound for δ^* which is added to as a constraint to the MIP problem. VHAGaR derives the initial binary hints from this PGD solution: each split ReLU is active or inactive at the returned perturbation, and VHAGaR sets the neuron’s binary to that activation state as a warm-start for the MIP. We modify this process for transfer mode by additionally deriving a hint value from N_{pre} ’s optimum analysis, which is a feasible point of N ’s MIP and may be close to N ’s own optimum.

From N_{pre} ’s analysis we read, for each split ReLU neuron i , its activation $v_i^{N_{\text{pre}}} \in \{0, 1\}$ (1 if active) and its pre-activation $\hat{z}_i^{N_{\text{pre}}}$. We compute the transfer probability $p_i \in [0, 1]$ from a heuristic interval $\tilde{I}_i$ that shifts $\hat{z}_i^{N_{\text{pre}}}$ by the weight-drift diff and intersects with N ’s tightened bound $[l_i^N, u_i^N]$:

$$I_i = [\hat{z}_i^{N_{\text{pre}}} + d_i^\downarrow, \hat{z}_i^{N_{\text{pre}}} + d_i^\uparrow], \quad \tilde{I}_i = I_i \cap [l_i^N, u_i^N], \quad (26)$$

Here $\hat{z}_i^{N_{\text{pre}}}$ is recovered from N_{pre} 's analysis. The transfer probability p_i is the fraction of $\tilde{I}_i$ on the side of zero consistent with N_{pre} 's decision ($\hat{z} \geq 0$ if active, $\hat{z} \leq 0$ if inactive),

$$p_i = \begin{cases} \frac{|\tilde{I}_i \cap (-\infty, 0]|}{|\tilde{I}_i|}, & v_i^{N_{\text{pre}}} = 0, \\ \frac{|\tilde{I}_i \cap [0, +\infty)|}{|\tilde{I}_i|}, & v_i^{N_{\text{pre}}} = 1, \end{cases} \quad (27)$$

where $|\cdot|$ denotes interval length, so p_i is the fraction of $\tilde{I}_i$ that still backs N_{pre} 's decision. We keep N_{pre} 's value when $p_i \geq 0.5$ and flip it otherwise:

$$\text{hint_val}(a_i) = \begin{cases} v_i^{N_{\text{pre}}}, & p_i \geq 0.5, \\ 1 - v_i^{N_{\text{pre}}}, & p_i < 0.5. \end{cases} \quad (28)$$

The hyper-adversarial attack has already populated Gurobi's warm-start $\text{Start}(a_i)$ on some binaries. We deliver $\text{hint_val}(a_i)$ through the same channel using a three-way consensus filter against any existing entry: (i) *fill* when PGD left $\text{Start}(a_i)$ unset, by setting $\text{Start}(a_i) \leftarrow \text{hint_val}(a_i)$; (ii) *leave* when PGD's value already matches $\text{hint_val}(a_i)$; (iii) *withdraw* when PGD's value disagrees with $\text{hint_val}(a_i)$, by clearing $\text{Start}(a_i)$ back to unset.

2 The `--geometric_intervals` Flag in `run.jl`

This section explains, in the simplest possible terms, the command-line flag `--geometric_intervals` in `run.jl`. It is the on/off switch for the relocation-aware bounds described in the Relocation-Aware Perturbed Intervals discussion above.

The one-sentence version. Turning the flag on makes the verifier compute *tighter* bounds for translation and rotation, so the solver (Gurobi) finishes the same job faster. It does *not* change the answer $\hat{\delta}_{ct}$ — only the time it takes to get there. The flag is `false` by default, which is exactly today's behaviour.

What problem it fixes. By default the verifier treats *every* perturbation as “add a little noise to each pixel,” where the noise is at most the budget ε in size. That is the right picture for an ℓ_∞ or brightness attack. For a translation or a rotation it is the *wrong* picture: the move does not add noise to a pixel, it *moves* pixel values around the image. Here ε means “shift by this many pixels” or “rotate by this angle,” which says nothing about how much a single pixel's brightness can change. So the default per-pixel box $[-\varepsilon, \varepsilon]$ is both meaningless and far too loose for these moves, and a loose bound makes the solver explore a needlessly large search space.

What the flag does instead. When the flag is on, the verifier uses the actual geometry of the move. A shift or rotation simply copies (or, for rotation, blends) a brightness value from one location to another, and every pixel value lives in $[0, 1]$. From that single fact we get a much tighter per-pixel rule for how much a pixel can change:

- a pixel that *receives* a value from elsewhere can change by at most ± 1 , i.e. $\Delta \in [-1, 1]$;
- a pixel that the move pushes *off the image* (and is therefore set to black) can only go down, i.e. $\Delta \in [-1, 0]$.

Internally the flag hands the solver the move’s exact “relocation map” ($T - I$) from Eq. (5), folds it into the first network layer (Eq. (6)), and from there propagates the tight per-pixel changes through the rest of the network with the same recurrence and ReLU clip as before (Eqs. (3) and (4)). The resulting bounds are typically much tighter than the generic $[-\varepsilon, \varepsilon]$, and that is what makes the solve faster.

A tiny example. Shift the image one step to the right by s pixels. Pixel d now shows whatever used to sit at pixel $d - s$, so its change is $x[d - s] - x[d]$, which lies in $[-1, 1]$. The default reasoning would report $[-s, s]$, a box that grows with the shift; the geometric bound stays inside $[-1, 1]$ no matter how large s is.

When it actually kicks in (and when it is ignored). The flag does something only when *all* of the following hold; otherwise `run.jl` prints a warning and silently falls back to the default (off) behaviour:

- the perturbation is `translation` or `rotation` — the only pixel-relocation moves; for anything else the flag is ignored;
- `--use_perturbed_intervals=true` — the flag only sharpens the clean-vs-perturbed coupling constraints, so with perturbed intervals turned off there is nothing for it to sharpen;
- the network’s first layer is fully connected (a `Flatten` feeding a dense layer), because the trick is folded into that first affine layer; a convolutional first layer falls back to the generic per-pixel bounds. For rotation the flag additionally applies to a single fixed angle.

It is interval-only: no zonotope is involved on this path.

Is it safe? Yes. The per-pixel rule above is a sound over-approximation of the true change, so no valid perturbation is ever excluded and the verified $\hat{\delta}_{c_t}$ is identical whether the flag is on or off (this is the Soundness argument above). The only thing that changes is the running time.

Reading the results. A run launched with the flag tags its output files with a `_geomInt` suffix. That way a geometric-interval run sits right next to its plain twin, and the two can be lined up and compared — same $\hat{\delta}_{c_t}$, hopefully a smaller solve time.

References

- [1] G. Singh, T. Gehr, M. Mirman, M. Püschel, and M. T. Vechev. Fast and Effective Robustness Certification. In *Advances in Neural Information Processing Systems 31, NeurIPS 2018*, pages 10825–10836, 2018.

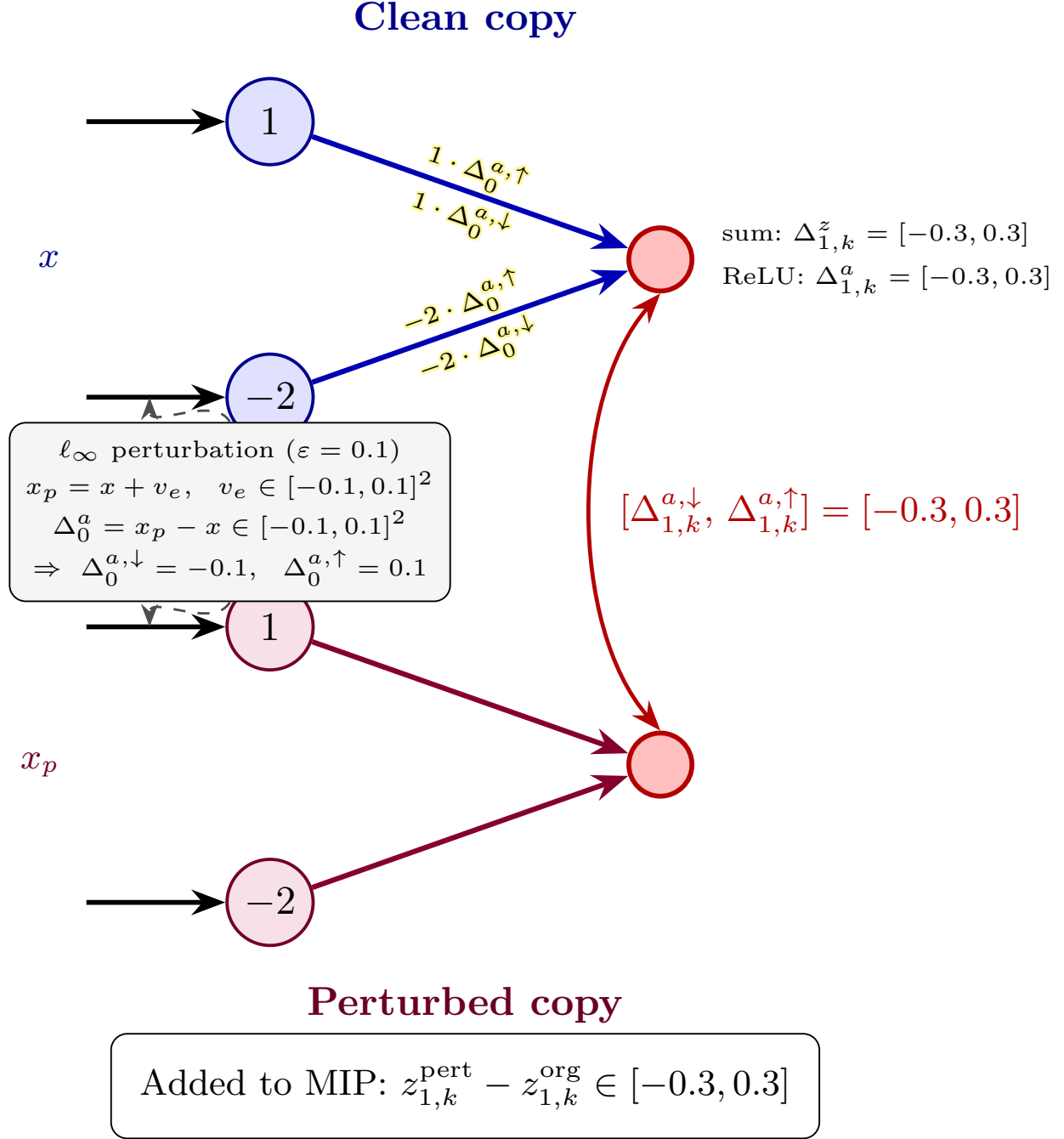


Figure 1: Worked example of perturbed-interval propagation for the neuron with weight row $(1, -2)$ under $\varepsilon = 0.1$. Each input neuron is labelled with its weight on the edge to neuron k , and each edge carries the up and down per-edge contributions $w \cdot \Delta_0^{a,\uparrow}$ and $w \cdot \Delta_0^{a,\downarrow}$; with $\Delta_0^a \in [-0.1, 0.1]$ per coordinate, summing them gives the pre-activation drift $[-0.3, 0.3]$, and the ReLU clip leaves $[\Delta_{1,k}^{a,\downarrow}, \Delta_{1,k}^{a,\uparrow}] = [-0.3, 0.3]$, which couples the two copies and is added to the MIP.